

C Programming

Ch.6 Conditional Statements

Operators | if / switch | while, do-while, for

Ch. 6

Conditional Statements

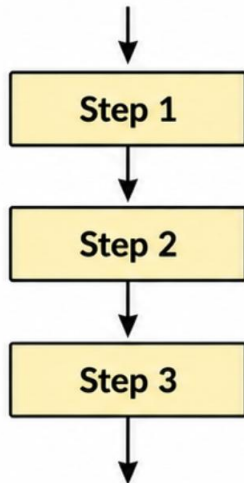
Conditional Statement

- If a program does not have a selection structure, the program will always repeat the same actions.
- Three types of control structures: sequential structure, selection structure, repeat structure.

Flowcharts show the basic ways a program executes.

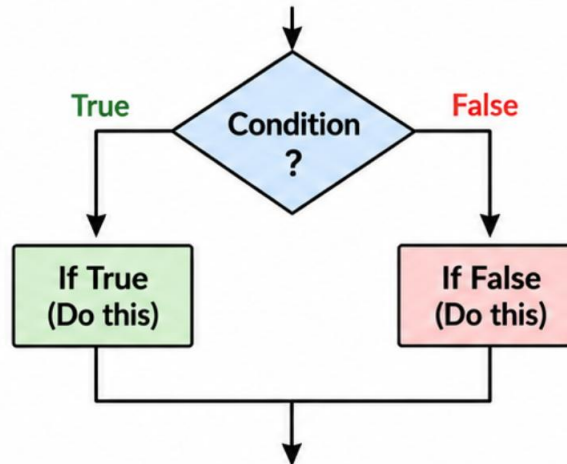
1. Sequential Structure

Steps are executed one by one in order.



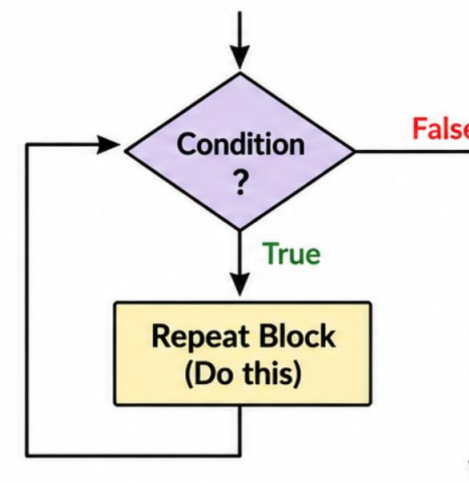
2. Selection Structure

A decision is made. Different paths are taken based on the condition.



3. Repeat Structure

A block of code is repeated while the condition is True.



1 Sequential: Do one thing after another.

2 Selection: Choose a path based on a condition.

3 Repeat: Repeat actions while a condition is True.

Conditional Statement: **if** statement

- **if statement**: in everyday life, there are many instances where decisions must be made based on conditions.

An if statement executes a block of code only when the condition is True.

1. SYNTAX

```
if ( condition ) statement;
```

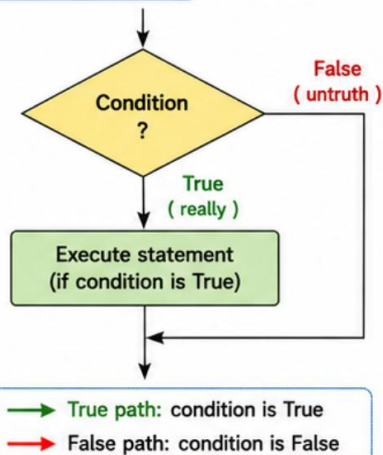
Example

```
if ( number > 0 )  
    printf("It is a positive number.\n");
```

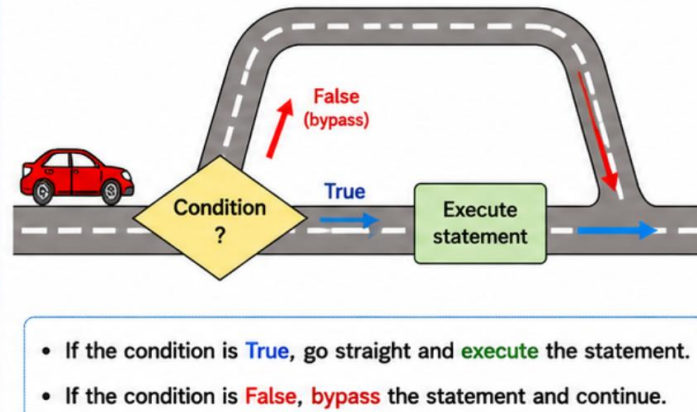
condition (expression)

statement (executed only if condition is True)

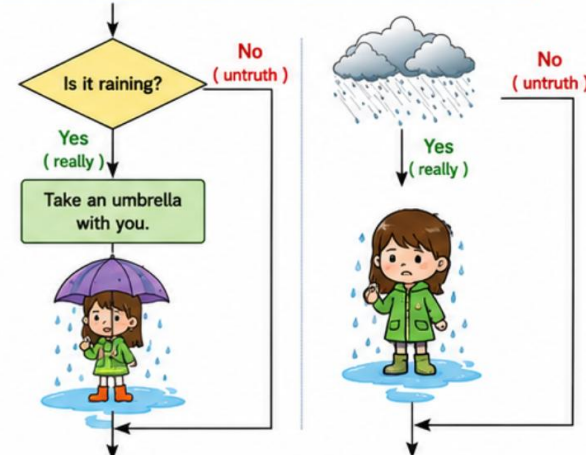
2. FLOWCHART



3. FLOW (ROAD MAP)



4. REAL-LIFE EXAMPLE



KEY POINTS



The statement is executed only if the condition is **True**.



If the condition is **False**, the statement is skipped.



if is the foundation of decision-making in C programs.

if Statement Example

```
#include < stdio.h >
int main( void )
{
    int number;
    printf ( "Enter an integer:" );
    scanf ( "%d" , &number);

    if ( number > 0 ) {
        printf ( "It is a positive number." );
    }
    printf ( "The entered value is %d.", number );

    return 0;
}
```

if the if-statement ends, the following line is always executed.

Compound Statement (Block)

- Compound statement: grouping statements using curly braces { }, also called a block.
- If the condition is true, all statements inside the block are executed:

```
if (score >= 60) { printf("You passed.\n"); printf("You can also get a scholarship.\n"); }
```

```
if (score >= 60) {
```

```
    printf("You passed.\n");
```

```
    printf("You can also get a scholarship.\n");
```

```
}
```


Conditional Statement: **if-else** Statement

An if-else statement executes one block when the condition is True, otherwise another block.

1. SYNTAX `if ( condition ) statement1; else statement2;`

Example

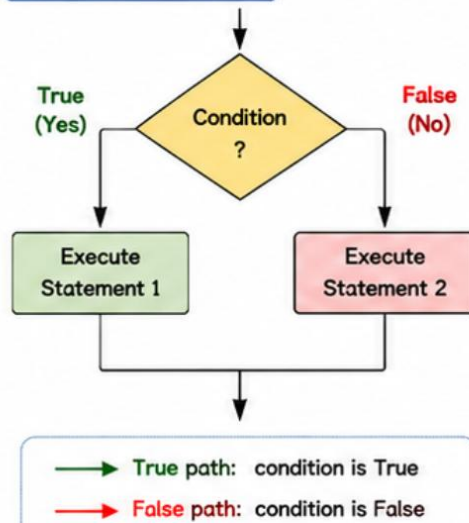
```
if ( number > 0 )  
    printf("It is a positive number.\n"); // statement1 (executed if condition is True)  
else  
    printf("Not a positive number.\n"); // statement2 (executed if condition is False)
```

condition (expression)

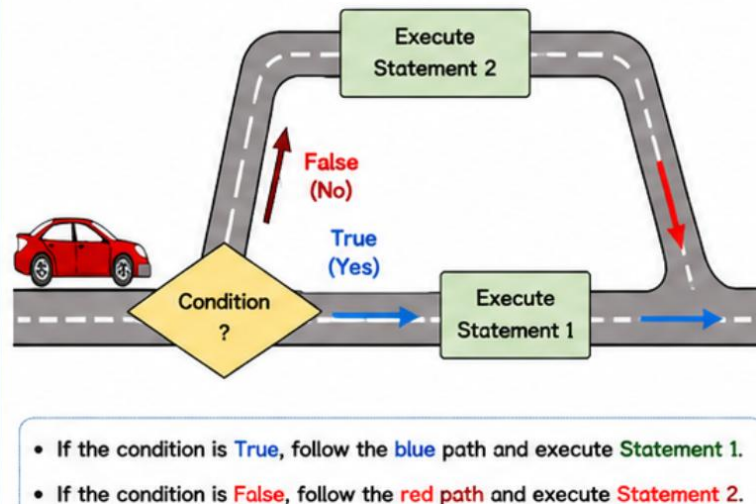
● — If the condition is True, statement1 is executed.

● — If the condition is False, statement2 is executed.

2. FLOWCHART

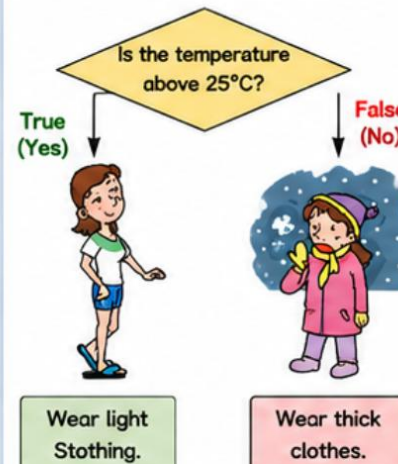


3. FLOW (ROAD MAP)

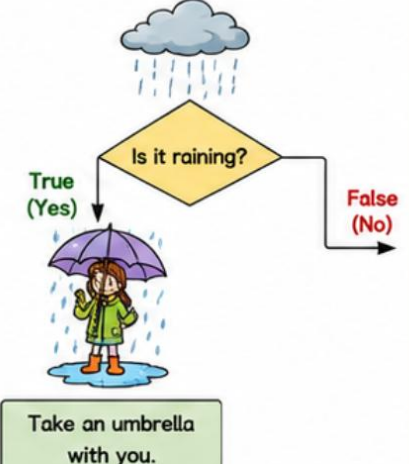


4. REAL-LIFE EXAMPLE

Example 1: Temperature



Example 2: Is it Raining?



KEY POINTS



The condition is tested.
If **True** → **Statement 1** runs.



If **False** → **Statement 2** runs.



if-else helps the program choose one of two possible actions.

Conditional Statement: **if-else** Statement

- Complex conditional expressions are also possible

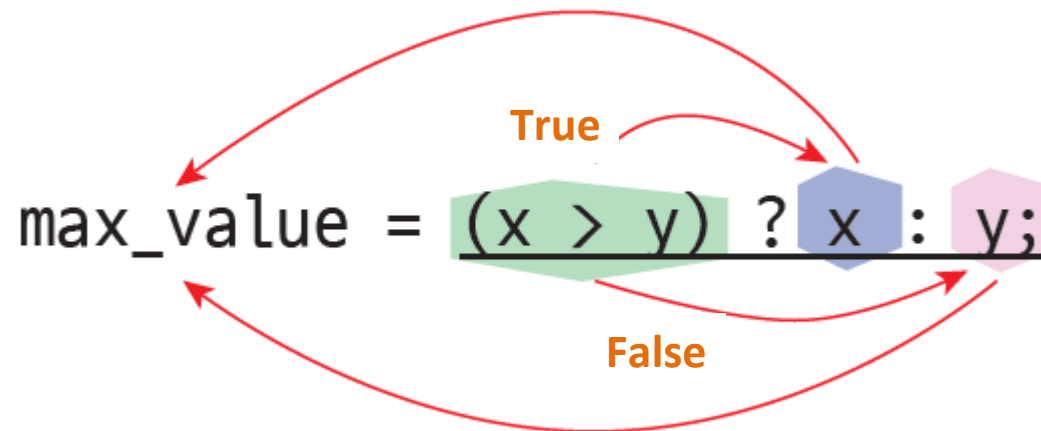
```
if ( score >= 80 && score < 90 )  
    grade = 'B' ;
```

```
if ( ch == ' ' || ch == '\n' || ch == '\t' )  
    white_space ++;
```

- Conditional **operator**: a simple if-else can be expressed with?

```
(score >= 60 ) ? printf( " Pass.\n" ) : printf( " Fail.\n" );
```

```
bonus = ( ( years > 30 ) ? 500 : 300 );
```



Coding Style

```
if (score >= 60)
    printf( "Passed\n" );
else
    printf( "Failed\n" );
```

```
if (score >= 60) {
    printf( "Passed\n" );
} else {
    printf( "Failed\n" );
}
```

```
if (score >= 60)
{
    printf( "Passed\n" );
}
else
{
    printf( "Failed\n" );
}
```

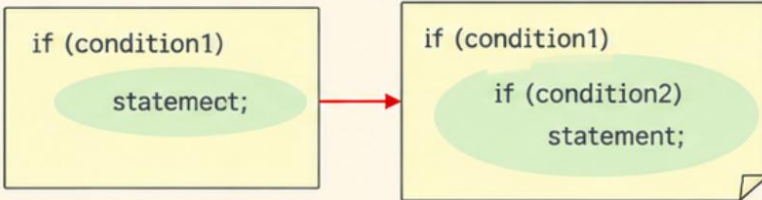
```
if (score >= 60) printf( "Passed\n" );
else printf( "Failed\n" );
```

Conditional Statement: Nested if and the Matching Problem

- Nested if: an **if statement** contains another **if statement**.

1. Statement inside a statement

- You can place an **if** statement inside another sentence. (**nested if**)



2. else matches the nearest if

- An **else** clause is paired with the **closest** unmatched **if**.



Key Points

- ✓ A statement can contain another **if** statement.
- ✓ An **else** belongs to the nearest **if**.
- ✓ Use **{ }** when you want an **else** to match a different **if** block.

Example 1: **else** matches the inner **if**

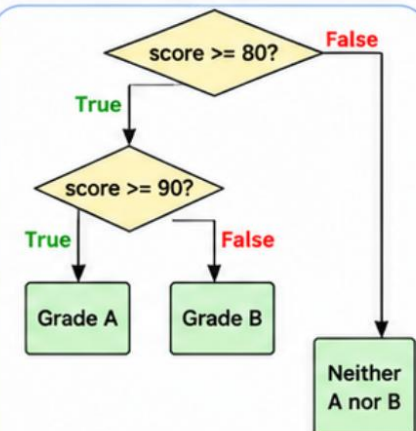
```
1 if (score >= 80)
2     if (score >= 90)
3         printf("Your grade is A.\n");
4     else
5         printf("Your grade is B.\n");
```

The **else** belongs to the nearest **if** (`score >= 90`).

Example 2: **else** matches the outer **if**

```
1 if (score >= 80) {
2     if (score >= 90)
3         printf("Your grade is A.\n");
4 }
5 else
6     printf("Your grade is neither an A nor a B.\n");
```

Braces **{ }** change which **if** the **else** matches.



Conditional Statement: Consecutive if (if – else if – else)

1. SYNTAX

Used when we need to check multiple conditions in order.

GENERAL FORM

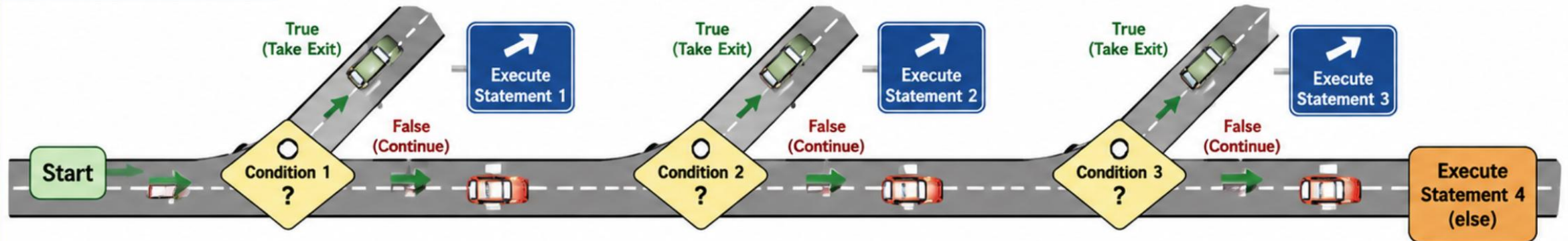
```
if ( condition1 )           statement 1;  
else if ( condition2 )     statement 2;  
else if ( condition3 )     statement 3;  
else                       statement 4;
```

HOW IT WORKS

- 1 If **condition 1** is true, **statement 1** is executed.
- 2 Otherwise, if **condition 2** is true, **statement 2** is executed.
- 3 Otherwise, if **condition 3** is true, **statement 3** is executed.
- 4 Otherwise, **statement 4** is executed.

2. FLOW (ROAD MAP)

The program checks conditions from top to bottom.



KEY POINTS

- ✓ Conditions are checked **in order** from top to bottom.
- ✓ The **first true condition** is executed, then the rest are skipped.
- ✓ If all conditions are false, the **else** block is executed.

Example: Determining Grades (if-else Chain)

```
#include < stdio.h >

int main( void )
{
    int score;
    printf ( "Enter your grade: " );
    scanf ( "%d" , &score);

    if (score >= 90)      printf ( "Passed: Grade A\n" );
    else if (score >= 80) printf ( "Passed: Grade B\n" );
    else if (score >= 70) printf ( "Passed: Grade C\n" );
    else if (score >= 60) printf ( "Passed: Grade D\n" );
    else                  printf ( "Failed: Grade F\n" );
    return 0;
}
```

Example: Character Classification

```
// Program to classify characters
#include <stdio.h>
int main( void )
{
    char ch ;
    printf ( "Enter a character: " );
    ch = getchar ();

    if ( ch >= 'A' && ch <= 'Z' )
        printf ( "%c is uppercase.\n" , ch );
    else if ( ch >= 'a' && ch <= 'z' )
        printf ( "%c is a lowercase letter.\n" , ch );
    else if ( ch >= '0' && ch <= '9' )
        printf ( "%c is a number.\n" , ch );
    else
        printf ( "%c is a miscellaneous character.\n" , ch );
    return 0;
}
```


switch Statement

Use switch when one variable can have many possible values.

1. SYNTAX

```
switch (controlled_expression)
{
    case value1:
        statement 1;
        break;
    case value2:
        statement 2;
        break;
    ...
    default:
        statement d;
        break;
}
```

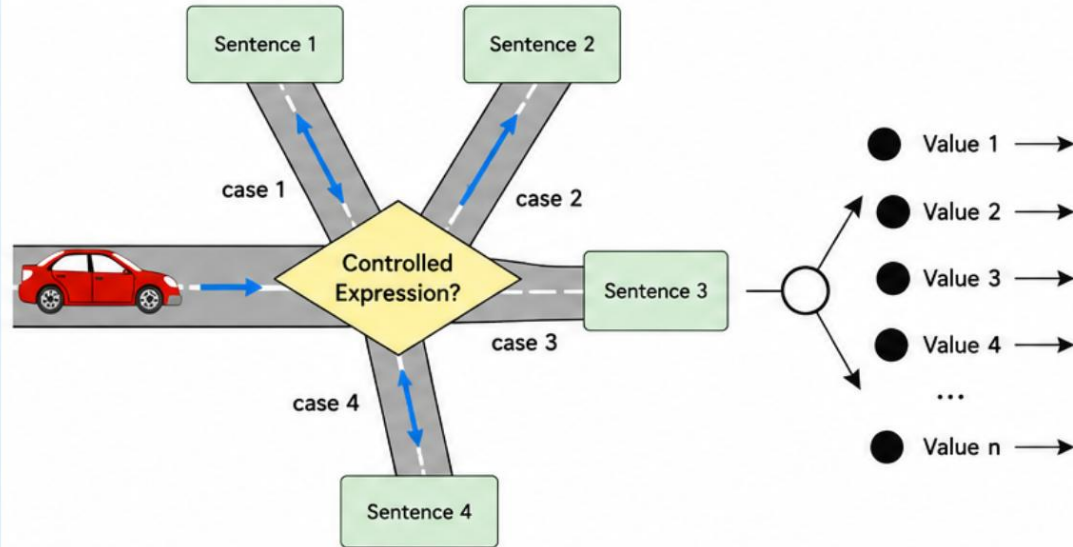
If the value of the control expression is value1, statement 1 is executed.

If the value of the control expression is value2, statement 2 is executed.

...

If no matching value is found, this is executed.

2. FLOW (ROAD MAP)



3. HOW IT WORKS

- 1 The **controlled expression** is evaluated.
- 2 Its value is compared with each **case** value.
- 3 If a match is found, that case's statements are executed until **break**;
- 4 If no match is found, the **default** case (if present) is executed.

4. EXAMPLE

```
int day = 3;
switch (day)
{
    case 1: printf("Mon\n"); break;
    case 2: printf("Tue\n"); break;
    case 3: printf("Wed\n"); break;
    default: printf("Invalid day\n");
}
```

If day = 3



Output: Wed

If day = 5



Output: Invalid day



KEY TAKEAWAYS



switch is ideal for multiple choices based on one variable.



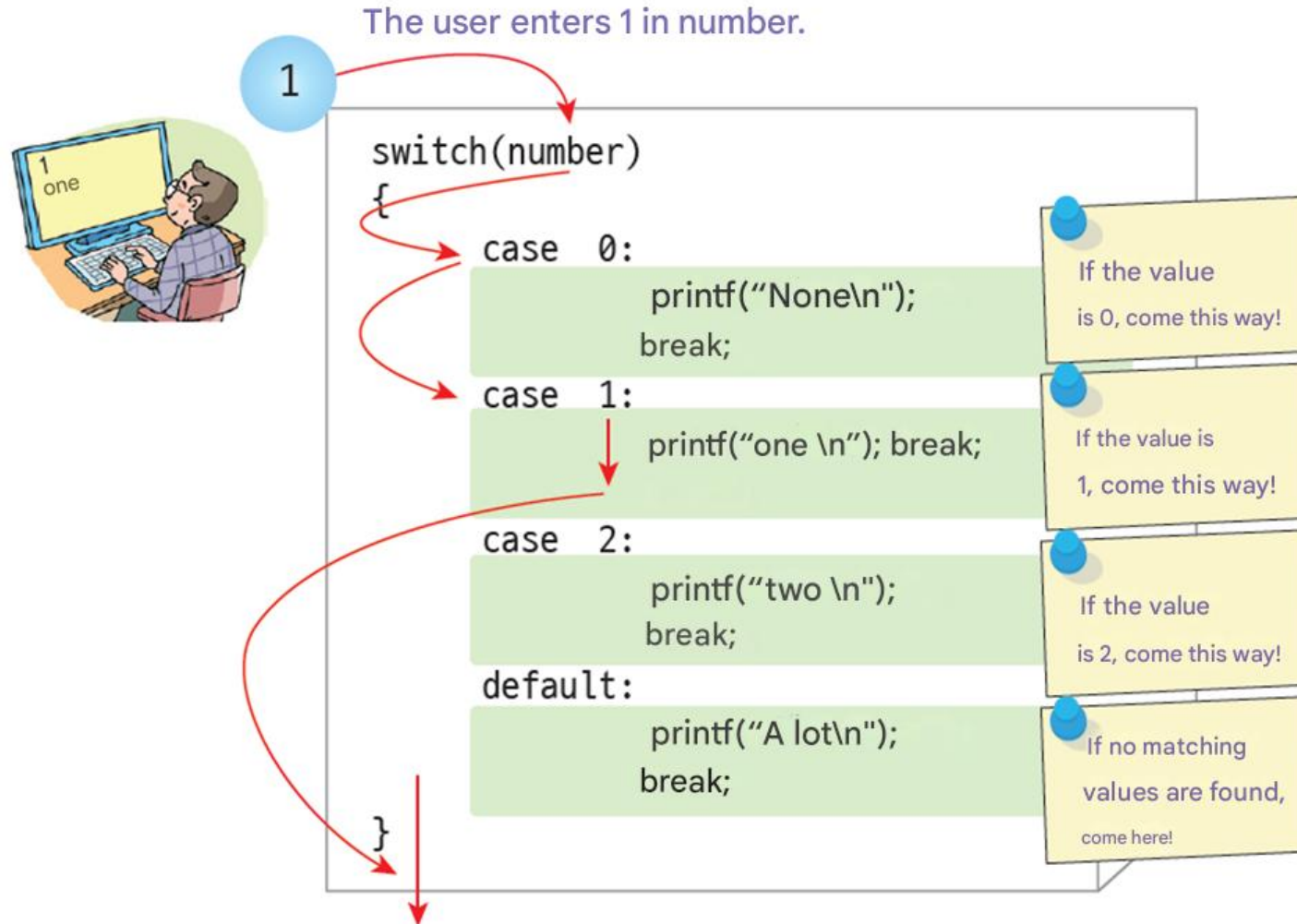
Use **break**; to stop execution after a case is matched.



Always include **default** to handle unexpected values.

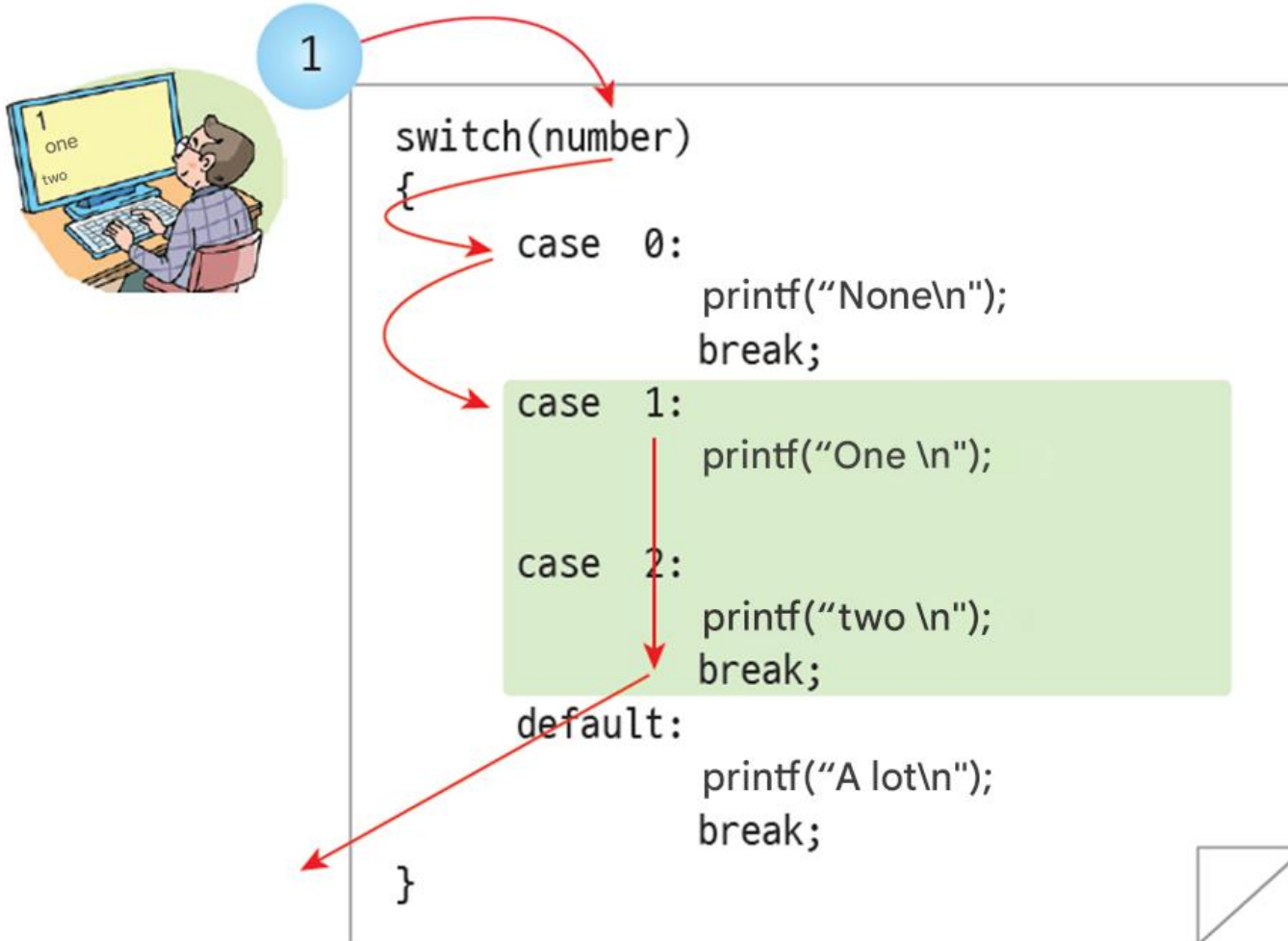
switch Statement: Case study 1

- User enter "1"



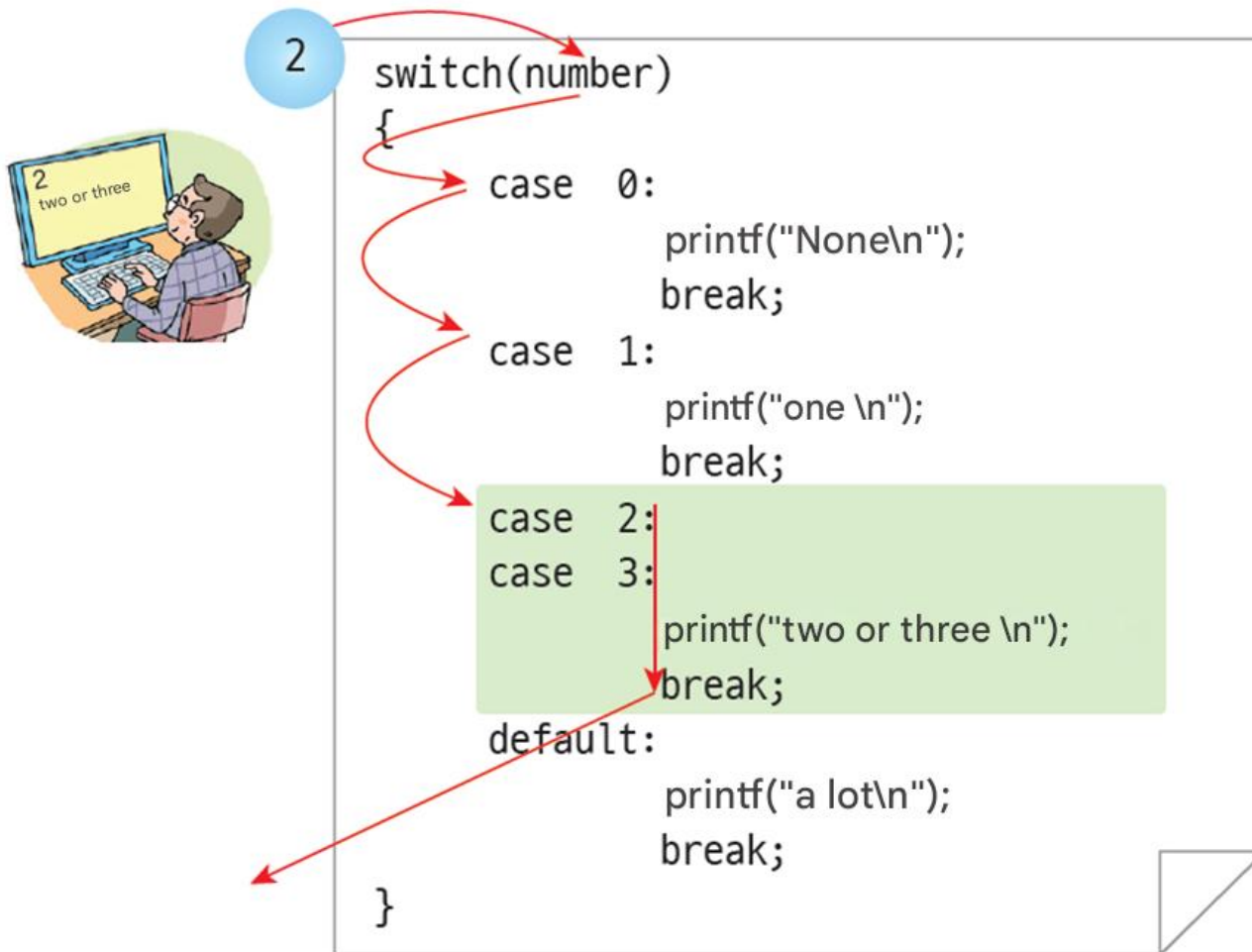
switch Statement: Case study 2

- If **break** is omitted



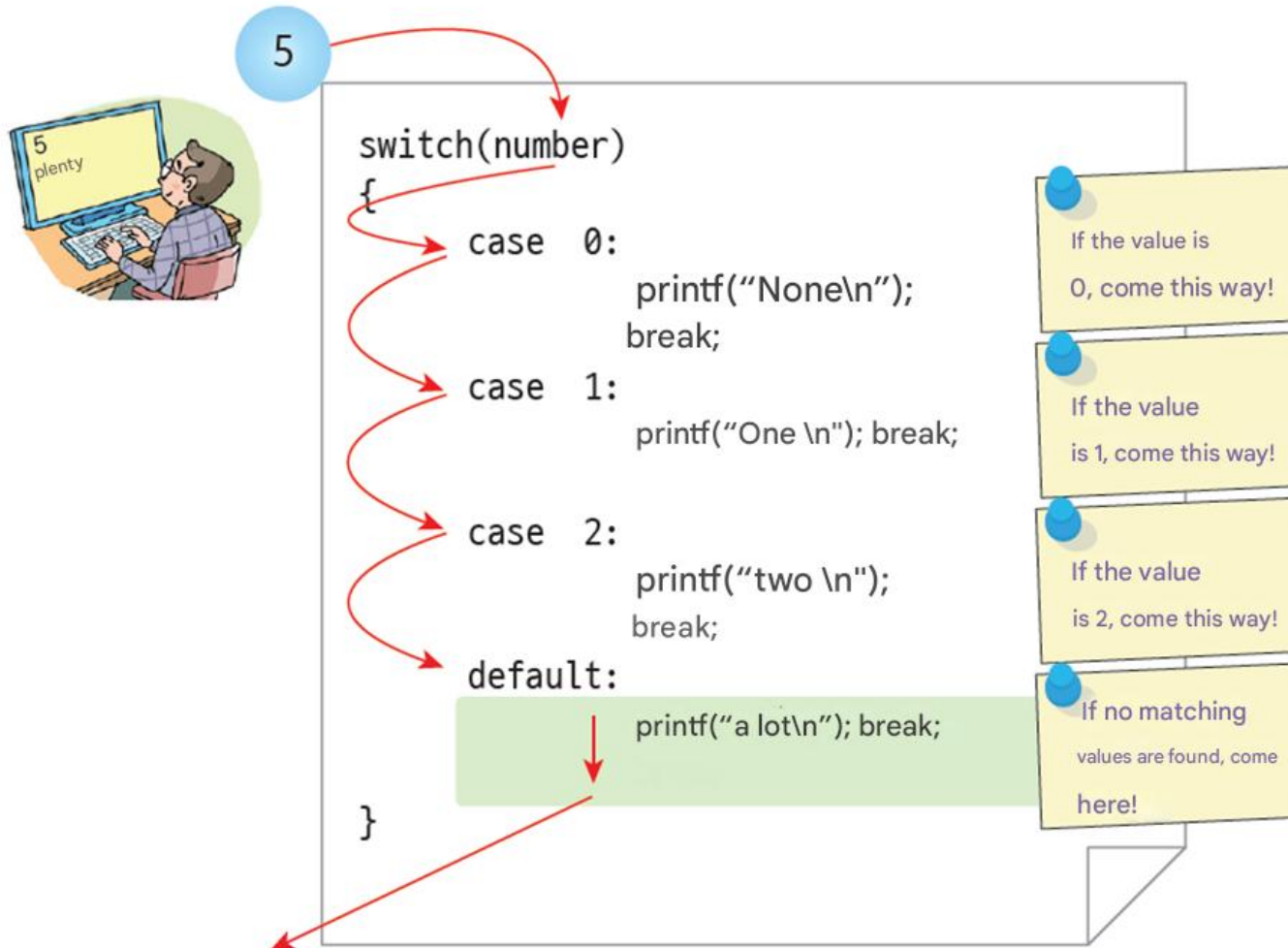
switch Statement: Case study 3

- Intentional omission of **break**: What is the intention?



switch Statement: Default

- User enter "5"



switch Statement: case label rule

Example

```
switch (number)
{
    case x:           // Variable cannot be used
        printf ( " Matches x . \n" );
        break ;

    case (x+2):       // Formulas containing variables
                    // cannot be used
        printf ( " Matches formula . \n" );
        break ;

    case 0.001:       // Real numbers cannot be used
        printf ( "error \n" );
        break ;

    case 'a' :        // OK! The character is can be used
        printf ( "character \n" );
        break ;

    case "abc":       // String cannot be used
        printf ( "string \n" );
        break ;
}
```

What Can and Cannot Be Used in case Labels?

Case Label	Example	Allowed?	Reason
Variable	case x:	✗ NO	Variables are not constant.
Expression with Variables	case (x+2):	✗ NO	Formulas containing variables are not allowed.
Real Number	case 0.001:	✗ NO	Real numbers (float, double) are not allowed.
Character (Constant)	case 'a':	✓ YES	Character constants are allowed.
String	case "abc":	✗ NO	Strings are not allowed.

IMPORTANT RULE



A case label must be a **CONSTANT INTEGER** OR **CONSTANT CHARACTER**.
(e.g., 1, 2, -5, 100, 'A', 'z', '\n')

QUICK SUMMARY

- ✓ Use integers (constants) and characters in case labels.
- ✓ Do not use variables, formulas with variables, real numbers, or strings.
- ✓ Always use break; to prevent fall-through.



switch Statement vs. if-else Statement



- ✓ A **switch** statement can always be rewritten using **if-else**.
- ✓ An **if-else** statement can be rewritten using **switch** only when the conditions compare one expression with constant values.

1. SWITCH → IF-ELSE (Always Possible)

SWITCH

```
switch (choice) {  
    case 1:  
        printf("Add\n");  
        break;  
    case 2:  
        printf("Delete\n");  
        break;  
    case 3:  
        printf("Exit\n");  
        break;  
    default:  
        printf("Invalid choice\n");  
}
```



EQUIVALENT IF-ELSE

```
if (choice == 1)  
    printf("Add\n");  
else if (choice == 2)  
    printf("Delete\n");  
else if (choice == 3)  
    printf("Exit\n");  
else  
    printf("Invalid choice\n");
```

- ✓ Each **case** in the **switch** becomes a condition in the **if-else**.

2. IF-ELSE → SWITCH (Only in Certain Cases)

IF-ELSE

```
if (choice == 1)  
    printf("Add\n");  
else if (choice == 2)  
    printf("Delete\n");  
else if (choice == 3)  
    printf("Exit\n");  
else  
    printf("Invalid choice\n");
```



EQUIVALENT SWITCH

```
switch (choice) {  
    case 1:  
        printf("Add\n");  
        break;  
    case 2:  
        printf("Delete\n");  
        break;  
    case 3:  
        printf("Exit\n");  
        break;  
    default:  
        printf("Invalid choice\n");  
}
```

- ✓ Works when all conditions compare the same expression (**choice**) with different **constant values** (1, 2, 3, ...).

switch Statement vs. if-else Statement

1. SWITCH vs IF-ELSE (RANGE EXAMPLE)

Using SWITCH

```
switch (score) {  
    case 100:    // OK  
    case 99:    // OK  
    case 98:    // OK  
    ...  
    case 90:  
        printf("It's an A.\n");  
        break;  
    ...  
}
```

✗ Cannot express a range like 90-100.
It is cumbersome to list every value.



Using IF

```
if (score >= 90 && score <= 100)  
    printf("It's an A.\n");
```



Easy and clear for
range conditions.

switch Statement vs. if-else Statement

2. SWITCH EXAMPLE (GRADING SYSTEM)

```
int iscore;
...
iscore = score / 10;    // Remainder is lost in integer division

switch (iscore) {
    case 9: grade = 'A'; break;    // 90-100 is A
    case 8: grade = 'B'; break;    // 80-89 is B
    case 7: grade = 'C'; break;    // 70-79 is C
    case 6: grade = 'D'; break;    // 60-69 is D
    default: grade = 'F'; break;    // 59 or less is F
}
```

How it works

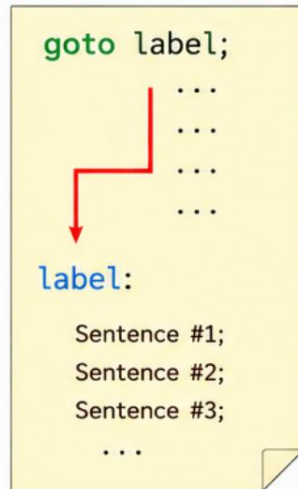
- 1 The expression (iscore) is evaluated.
- 2 Its value is compared with each case label.
- 3 If a match is found, that case is executed.
- 4 If no match is found, default is executed.

goto statement

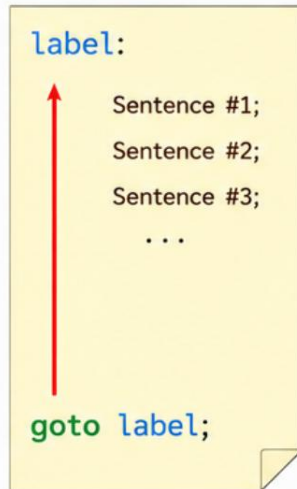
goto transfers control to a **labeled statement** within the same function.

1. HOW IT WORKS

Forward looking



Retrospective reference



A **label** is an identifier followed by a colon (:). It marks the place to which control can be transferred.

2. EXAMPLE PROGRAM

```
// Multiplication table output program
#include <stdio.h>

int main(void)
{
    int i = 1;

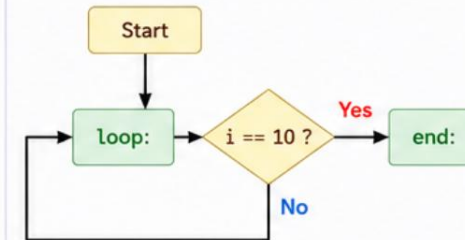
loop:
    printf("%d * %d = %d \n", 3, i, 3 * i);
    i++;
    if (i == 10) goto end;
    goto loop;

end:
    return 0;
}
```

3. OUTPUT

```
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
```

Flow of control

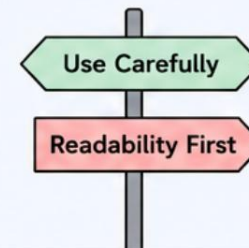


4. NOTES

- ✓ Use **goto** carefully. It can make code hard to read.
- ✓ It is still used in some situations (error handling, cleanup, breaking out of nested loops, etc.).
- ! Do not jump outside the current function.

5. KEY POINTS

- **goto** jumps to a **label** in the same function.
- **Labels** mark target statements.
- Control can move forward or backward.
- Use it only when it improves program clarity.



Lab: Arithmetic Calculator (if-else & switch)



This program reads two integers and an operator (+, -, *, /), performs the calculation, and prints the result.

1. Using if-else

```
#include <stdio.h>

int main(void)
{
    int num1, num2;
    char op;

    printf("Enter two numbers: ");
    scanf("%d %d", &num1, &num2);

    printf("Enter operator (+, -, *, /): ");
    scanf(" %c", &op);

    if (op == '+')
        printf("Result = %d\n", num1 + num2);
    else if (op == '-')
        printf("Result = %d\n", num1 - num2);
    else if (op == '*')
        printf("Result = %d\n", num1 * num2);
    else if (op == '/')
        printf("Result = %d\n", num1 / num2);
    else
        printf("Invalid operator\n");

    return 0;
}
```



2. Using switch

```
#include <stdio.h>

int main(void)
{
    int num1, num2;
    char op;

    printf("Enter two numbers: ");
    scanf("%d %d", &num1, &num2);
    printf("Enter operator (+, -, *, /): ");
    scanf(" %c", &op);

    switch (op)
    {
        case '+':
            printf("Result = %d\n", num1 + num2);
            break;
        case '-':
            printf("Result = %d\n", num1 - num2);
            break;
        case '*':
            printf("Result = %d\n", num1 * num2);
            break;
        case '/':
            printf("Result = %d\n", num1 / num2);
            break;
        default:
            printf("Invalid operator\n");
            break;
    }

    return 0;
}
```

Q & A

Lab & Quiz: right after this - see you in the practice session!